GSLC - Stack & Queue

Name : Wilbert Chandra

NIM : 2602113666

Question

1. What is the different between stack and queue data structure?
2. Give example real implementation of stack and queue (each 1 example)
3. Pg. 251-252
    a. What do you understand by stack overflow and stack underflow?
    b. Differentiate between an array and a stack.
    c. How does stack implemented using a linked list differ from a stack implemented using an array?
    d. Why are parentheses not required in postfix/prefix expressions?
    e. Explain how stacks are used in a non-recursive program?
    f. What do you understand by a multiple stack? How is it useful?
    g. Explain the terms infix expressions, prefix expression, and postfix expression. Convert the following infix expressions to their postfix equivalents :
        i. A – B + C
        ii. A * B + C / D
        iii. (A – B) + C * D / E – C
        iv. (A * B) + (C / D) – (D – E)
        v. ((A – B) + D / ((E + F) * G))
        vi. (A – 2 * (B + C) / D * E) + F
        vii. 14 / 7 * 3 – 4 + 9 / 2
    h. Convert the following infix expressions to their postfix equivalents :
        i. A – B + C
        ii. A * B + C / D
        iii. (A – B) + C * D / E – C
        iv. (A * B) + (C / D) – (D – E)
        v. ((A – B) + D / ((E + F) * G))
        vi. (A – 2 * (B + C) / D * E) + F
        vii. 14 / 7 * 3 – 4 + 9 / 2
    i. Find the infix equivalents of the following postfix equivalents :
        i. A B + C * D -
        ii. ABC * + D -
    j. Give the infix expression of the following prefix expressions.
        i. * – + A B C D
        ii. + – a * B C D
    k. Convert the expression given below into it's corresponding postfix expression and then evaluate it. Also write a program to evaluate a postfix expression.
        i. 10 + ((7 – 5) + 10) / 2
    l. Write a function that accepts two stacks. Copy the contents of first stack in the second stack. Note that the order of elements must be preserved. (Hint: use a temporary stack)
    m. Draw the stack structure in each case when the following operations are performed on an empty stack.

i. Add A, B, C, D, E, F
ii. Delete 2 letters
iii. Add G
iv. Add H
v. Delete four letters.
vi. Add I

4. Pg. 277
   a. Draw the queue structure in each case when the following operations are performed on an empty queue.
      i. Add A, B, C, D, E, F
      ii. Delete 2 letters
      iii. Add G
      iv. Add H
      v. Delete four letters.
      vi. Add I
   b. Consider the queue given below which has FRONT = 1 and Rear = 5

	A	B	C	D	E				

Now perform the following operations on the queue :

1. Add F
2. Delete 2 Letters
3. Add G
4. Add H
5. Delete 4 letters
6. Add I

Answers

1. The difference between stack and queue data structure is
    a. Stack
        i. Last in – First out Order, meaning the data that the last data that got stored will be the first data that are outputted.
        ii. Support push (Insert data to the top of the stack), pop (Delete data on the top of the stack), peek (See the data on the top of the stack), isEmpty (Check if the stack is empty or not) Operation.
        iii. Can be used to Reverse a list, check parentheses, convert infix to postfix, evaluate postfix, recursion, undo-redo function, and many more.
        iv. When implementing with an array, the stack data structure only needs the top of the array.
    b. Queue
        i. First in - First out Order, meaning the first data stored will also be the first outputted data.
        ii. Support enqueue (insert data to the rear of the queue), dequeue (delete data on the front of the queue), peek (see the data on the front of the queue), isEmpty (check if the queue is empty or not).
        iii. Can be used as a way to make a waiting list for a single shared resource, to transfer data asynchronously, as a playlist for songs or video, to handle interrupts
        iv. When implementing with an array, the queue data structure needs the array's front and rear.
        v. Have 3 types, which are Circular queue, Priority queue, and Double-Ended queue.

2. The real implementation of stack and queue is
    a. Stack :

        Undo–redo function in most software, for example, Microsoft word. When you type or delete a string of text, the current state is then pushed onto the top of the stack. when you undo then the newest state is popped from the top of the stack. When you redo then the state that's have been undone is pushed back to the stack.

    b. Queue
        Restaurant System. In most restaurants, the customer will be served on a first come first served basis. Which is the same as queueing the customer. When the customer came then, they will be enqueued to the rear of the queue, similarly when the customer is served then they will be dequeued from the front of the queue.

3. Below
    a. Stack Overflow and Stack Underflow is
        i. Stack Overflow: An error that occurred when a program tries to push data to a full stack. When this happens, the program may crash or show some weird behavior like overwriting data.
        ii. Stack Underflow: An error that occurred when a program tries to pop data from an empty stack. When this happens, the program may crash or show some weird behavior like accessing invalid memory.

    b. The Difference between an array and a stack is

i. Array
    1. Data can be accessed by index.
    2. Allocate Memory Statically.
    3. Can only contain data of the same data type.
    4. Can do both linear and binary searches.
    5. Data can be searched randomly.
    6. Has no pointer.

ii. Stack
    1. Data can only be accessed when it's on the top of the stack.
    2. Can allocate memory dynamically by using a linked list.
    3. Can contain multiple data types at the same time.
    4. Can only do a linear search.
    5. Data can't be searched randomly.
    6. Has 1 pointer to the top element.

c. Stack that's implemented using a linked list differs from a stack that's implemented using an array by its way of storing data dynamically for linked list and statically for array, and linked list capabilities to store data with different data types at the same time while array can't.

d. Parentheses are not required in postfix and prefix expressions because in infix expression, parentheses is used to indicate an order of operations. But, with prefix and postfix expression the order of operations is indicated by the placement of operator. "Add Examples"

e. Stacks are used in non-recursive programs the same way it's used in recursive programs "Add Examples"

f. Multiple Stack is a way to create multiple stack but also conserve memory by sharing a space between the multiple stacks. Multiple stack is usually used in a program that need multiple stacks but have concern with its memory usage thus resorting to a multiple stacks.

g. The terms infix, postfix, and prefix expressions is
    i. Infix Expression is an expression where the operator is in the middle of the operands. For example, 2 + 3.
    ii. Postfix Expression is an expression where the operator is after the operands. For example, 2 3 +.
    iii. Prefix Expression is an expression where the operator is before the operands. For example, + 2 3.

    The postfix equivalents of the infix expressions is

    1. A B – C +
    2. A B * C D / +
    3. A B – C D * E / + C -
    4. A B * C D / + D E - -
    5. A B – D E F + G * / +
    6. A 2 B C + * D / E * - F +

7. 14 7 / 3 * 4 9 2 / + -

h. The postfix equivalents of the infix expressions is
   i. A B – C +
   ii. A B * C D / +
   iii. A B – C D * E / + C -
   iv. A B * C D / + D E - -
   v. A B – D E F + G * / +
   vi. A 2 B C + * D / E * - F +
   vii. 14 7 / 3 * 4 9 2 / + -

i. The infix equivalents of the postfix equivalents is
   i. (A + B) * C – D
   ii. ABC * + D - = A + B * C – D

j. The infix equivalents of the prefix expression is
   i. ((A + B) – C) * D
   ii. a - B * C + D

k. The postfix expression of the given expression is
   i. Postfix expression : 10 7 5 – 10 + 2 / +
   ii. Evaluation :
      1. Program Code :

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

float stack[MAX];
int top = -1;

float char_to_float(char a) {
    float value = (float) a - '0';
    return value;
}

void push(float value) {
    if (top == MAX -1)
    {
        printf("Stack Overflow\n");
        return;
    }

    top++;
    stack[top] = value;

    // printf("Top %.2f\n", stack[top]);

    return;
}

float pop() {
    float value = 0;

    if (top == -1)
    {
        printf("Stack Underflow\n");
        return 0;
    }

    value = stack[top];
    top--;

    return value;
}

float evalPostfix(char expression[]) {
    int i = 0;
    float x, y, value;
```

2. Test Run :

```
Enter a Postfix Expression : 9 3 4 * 8 + 4 / -
The Evaluated Value of the postfix expression is = 4.00
```

l. This is a function that I made

```
stack *copyStacks(stack *stk1, stack *stk2) {
    stack *temp = NULL;

    while (!isEmpty(stk1)) {
        temp = push(temp, stk1->data);
        stk1 = pop(stk1);
    }

    while (!isEmpty(temp))
    {
        stk2 = push(stk2, temp->data);
        temp = pop(temp);
    }

    return stk2;
}
```

It works bu copying the first stack to the temp stack until the 1st stack is empty then copying the temp stack to the 2nd stack until the temp stack is empty.

m. The Stack Structure in each case is

F	D	G	H	B	I
E	C	D	G	A	B
D	B	C	D		A
C	A	B	C		
B		A	B		
A			A		

4. Pg. 277

a. The queue structure that will be made is

A	B	C	D	E	F				
		C	D	E	F				
		C	D	E	F	G			
		C	D	E	F	G	H		
						G	H	I	

b. The operation will be :

	A	B	C	D	E				
	A	B	C	D	E	F			
			C	D	E	F			
			C	D	E	F	G		
			C	D	E	F	G	H	
							G	H	
							G	H	I